

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid1, pid2;

    pid1 = fork();

    if (pid1 == 0)
    {
        printf("Child 1\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());

        sleep(2);

        printf("Child 1 Finished\n");
        exit(0);
    }

    pid2 = fork();

    if (pid2 == 0)
    {
        printf("Child 2\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());

        sleep(4);

        printf("Child 2 Finished\n");
        exit(0);
    }

    printf("\nParent Process\n");
    printf("Parent PID : %d\n", getpid());
}
```

```

    wait(NULL);
    printf("One child completed using wait().\n");

    waitpid(pid2, NULL, 0);
    printf("Second child completed using waitpid().\n");

    printf("Parent Finished\n");

    return 0;
}

```

Output:

```

uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical
Parent Process
Parent PID : 884
Child 2
PID : 886
Child 1
PPID : 884
PID : 885
PPID : 884
Child 1 Finished
One child completed using wait().
Child 2 Finished
Second child completed using waitpid().
Parent Finished

```

Zombie_ fixed.c:

```

GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("PID : %d\n", getpid());

        exit(0);
    }
    else
    {
        printf("Parent Process\n");
        printf("PID : %d\n", getpid());

        wait(NULL);

        printf("Child collected successfully.\n");
        printf("No Zombie Process.\n");
    }

    return 0;
}

```

Output:

```

uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical$
Parent Process
PID : 963
Child Process
PID : 964
Child collected successfully.
No Zombie Process.

```

Zombie_Process.c

```
GNU nano 8.7.1                                Zombie_Process.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        printf("Fork Failed\n");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("PID : %d\n", getpid());
        printf("Child exiting...\n");
        exit(0);
    }
    else
    {
        printf("Parent Process\n");
        printf("PID : %d\n", getpid());
        printf("Sleeping for 20 seconds...\n");

        sleep(20);

        printf("Parent exiting.\n");
    }

    return 0;
}
```

GNU nano 8.7.1 interface with various keyboard shortcuts listed at the bottom: ^G Help, ^O Write Out, ^F Where Is, ^K Cut, ^T Execute, ^C Location, ^U Undo, ^A Set Mark, ^M To Bracket, ^X Exit, ^R Read File, ^N Replace, ^V Paste, ^J Justify, ^_ Go To Line, ^E Redo, ^G Copy, ^B Where Was.

Output:

```
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Practical-04$ nano Zombie_Process.c
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Practical-04$ gcc Zombie_Process.c -o Zombie_Process
uday@DESKTOP-47L1GTF:~/2520090089_OSSP/2520090089_Practical/Practical-04$ ./Zombie_Process
Parent Process
PID : 1026
Sleeping for 20 seconds...
Child Process
PID : 1027
Child exiting...
Zombie_Process.cParent exiting.
```